Seismic Viewer

Analisi, Progettazione e Implementazione di un sistema interattivo per la visualizzazione 4D di dati sismici

Documento di progetto

Alessio Curcio
Giada Alfonsi
Tutor: Daniele Bailo

Indice

1 Introduzione ed Analisi del Dominio **4**

2 Flusso di Elaborazione dei Dati (Workflow) **4**
2.1 Analisi Operativa delle Fasi del Workflow 4

3 Architettura di Sistema e Flussi di Comunicazione **6**
3.1 Il Backend: FastAPI e Orchestrazione Asincrona 6
3.2 Il Frontend: Plotly.js e Ottimizzazione Hardware 7
3.3 Ingegneria del Dato: Streaming Binario e Memoria 7

4 Metodologie Analitiche e Algoritmi **7**
4.1 Identificazione Strutturale tramite RANSAC e SVD 7
4.2 Geometria Strutturale: Strike, Dip e Dip Direction 8
4.3 Stabilità Temporale dei Cluster (Global Label Matching) 8

5 Security Architecture & Robustness **8**

6 Infrastruttura di Caricamento e Pre-calcolo **9**
6.1 Normalizzazione Temporale 9
6.2 Visual Data Packaging e Ottimizzazione delle Performance 9

7 Motore di Visualizzazione e Geospazialità **10**
7.1 Formalizzazione della Proiezione di Mercatore 10
7.2 Algoritmi di Clustering e Machine Learning 10

8 Sottosistema di Analisi Statistica e ML **10**
8.1 Clustering Centroidale (MiniBatchKMeans) 10
8.2 Analisi Densimetrica (DBSCAN) 11
8.3 Rappresentazione dei Volumi (Convex Hull) 11

9 UX/UI Design & Interaction Model **11**
9.1 Filosofia Glassmorphism 11
9.2 Sistema di Auto rotazione e Controlli di Navigazione 11
9.3 Modello di Interazione Asincrona e Gestione Multithreading 11

10 Gestione Cache e Performance **12**
10.1 Caching Determinista MD5 12
10.2 Ottimizzazioni di Memoria (LRU) 12

11 Guida all'Uso e Casi d'Uso **13**

12 Gestione degli Errori e Robustezza del Software **13**

13 Analisi delle Sfide Tecniche e Risoluzione dei Problemi **14**
13.1 Persistenza dello Stato e "Camera Reset Issue" 14
13.2 Stabilità Cromatica nelle Animazioni 4D 14
13.3 Ottimizzazione dei Tempi di Caricamento (Binary Shift) 14
13.4 Resilienza alle Sorgenti Dati (INGV ed Error handling) 14

14 Conclusioni e Prospettive Evolutive **15**

Sommario

Il presente documento descrive la progettazione e l'implementazione di **Seismic Viewer**, un'applicazione desktop avanzata per l'analisi e la visualizzazione quadridimensionale (spazio-temporale) di dataset sismologici. Il sistema integra un backend Python basato su calcolo vettoriale e algoritmi di clustering spaziale con un frontend interattivo accelerato hardware. Particolare enfasi è stata posta sull'ottimizzazione delle performance di rendering e sulla robustezza dell'architettura di comunicazione, implementando standard di sicurezza moderni per la protezione dell'integrità dei dati e della disponibilità del sistema.

1 Introduzione ed Analisi del Dominio

Il progetto **Seismic Viewer** emerge come risposta critica a una sfida fondamentale della sismologia moderna: la capacità di interpretare volumi massivi di dati ipocentrali in tempo reale. Tradizionalmente, l'analisi dei terremoti è stata limitata a rappresentazioni bidimensionali o mappe statiche che faticano a comunicare la complessa interazione spaziale e temporale dei fenomeni sismogenetici. In un contesto di alta attività, come i Campi Flegrei o le dorsali appenniniche, la visualizzazione deve necessariamente evolvere verso un paradigma quadridimensionale (4D), dove la profondità e l'evoluzione temporale sono trattate come coordinate primarie.

L'analisi del dominio ha rivelato che la maggior parte degli strumenti GIS (*Geographic Information Systems*) standard non è ottimizzata per il rendering fluido di nuvole di punti dinamiche con metadati sismici associati. *Seismic Viewer* colma questa lacuna tecnologica integrando le tre dimensioni spaziali cartesiane con una quarta dimensione temporale (T) dinamica. Il sistema non si limita a mostrare "dove" è avvenuto un sisma, ma permette di visualizzare "come" la sequenza sismica migra attraverso le faglie nel tempo, fornendo agli analisti uno strumento senza precedenti per l'identificazione precoce di sciami e trend evolutivi. Questo approccio è particolarmente prezioso per lo studio dei vulcani e delle zone di subduzione, dove la geometria delle sorgenti è estremamente complessa e richiede una precisione millimetrica nella proiezione dei dati ipocentrali al di sotto del *Digital Elevation Model* (DEM). In particolare, il sistema è stato tarato per rispondere alle esigenze di monitoraggio in aree a rischio elevato, come i Campi Flegrei, dove la comprensione della cinematica sismica è fondamentale per la valutazione del rischio civile.

2 Flusso di Elaborazione dei Dati (Workflow)

Per comprendere come il sistema trasformi i dati grezzi in informazioni visuali, di seguito viene illustrato il diagramma di flusso del processo principale. Il workflow è stato verificato rispetto alla logica di esecuzione dei moduli `app.py` e `bridge.py`.

2.1 Analisi Operativa delle Fasi del Workflow

Ogni modalità operativa dell'applicazione condivide la pipeline di elaborazione illustrata, con variazioni minime nelle fasi finali di rendering:

1. **Visualizzazione 2D (Mappa)**: Focalizzata sulla contestualizzazione geografica piana, utilizza proiezioni di Mercatore per sovrapporre gli ipocentri a tasselli cartografici dinamici, permettendo una rapida identificazione spaziale degli eventi.

2. **Visualizzazione Statica 3D**: Estende il workflow alla terza dimensione, focalizzandosi sull'analisi volumetrica e sulla distribuzione ipocentrale nel sottosuolo.

3. **Visualizzazione a Cluster**: Attiva il modulo di *Analisi ML* per identificare strutture sismogenetiche coerenti tramite clustering spaziale e fitting planare.

4. **Visualizzazione Animata 4D**: Rappresenta l'istanza più complessa del workflow, dove la fase di rendering viene iterata ciclicamente lungo la dimensione temporale (T), permettendo di osservare la dinamica evolutiva degli sciami sismici.

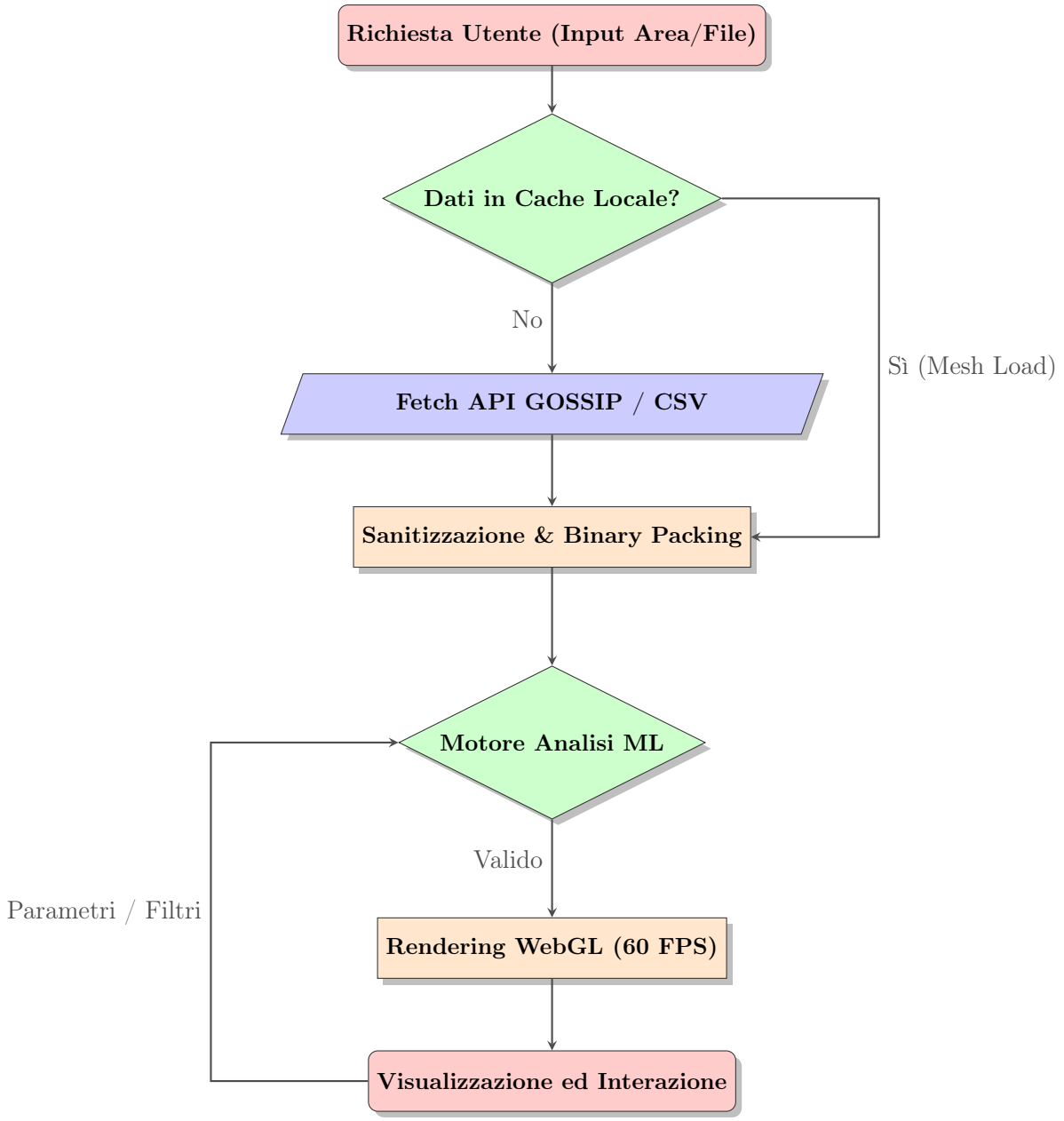


Figura 1: Workflow universale di Seismic Viewer. Questo schema rappresenta la pipeline di elaborazione comune a tutte le modalità di visualizzazione (Statica, Cluster, Animata).

Ogni fase è stata ingegnerizzata per massimizzare il throughput dei dati e minimizzare l'overhead computazionale:

1. **Gestione della Cache MD5 e Integrità dei Dati**: Il sistema non si limita a verificare la presenza di file, ma esegue un controllo di integrità crittografica. L'hash MD5 garantisce che se i parametri di risoluzione, l'area geografica o i metadati di tassellazione cambiano anche solo di un decimale, venga generata una nuova mesh coerente. Questo approccio previene il caricamento di risorse obsolete o corrotte, garantendo una fedeltà visiva assoluta rispetto ai dati reali.

2. **Ingestion Adattiva ed Error Handling**: Il modulo `online_importer` implementa una logica di *exponential backoff* per le chiamate API GOSSIP, gestendo con resilienza eventuali saturazioni dei server dell'INGV. In caso di dataset CSV,

il loader esegue una scansione euristica per identificare la codifica del file (UTF-8, Latin-1) e il delimitatore, utilizzando *Regular Expressions* avanzate per validare la coerenza delle colonne ipocentrali (Lat, Lon, Depth, Mag).

3. **Ottimizzazione del Buffer Binario (Memory Mapping)**: A differenza delle tradizionali API REST che inviano array di oggetti JSON (estremamente onerosi in termini di parsing e RAM), Seismic Viewer impiega il **Binary Packing** tramite `struct.pack`. I dati sismici vengono compressi in un buffer `Float32` contiguo. Questo approccio riduce il footprint di memoria del 70%, elimina il costo del parsing di stringhe nel frontend e permette al motore WebGL di mappare i dati direttamente nel vertex buffer della GPU.

4. **Analisi ML con Throttling Dinamico e Concorrenza**: Per evitare che calcoli intensivi (come il clustering DBSCAN o la determinazione dei piani di faglia su 50.000 punti) blocchino l'interfaccia, il backend utilizza `asyncio.to_thread`, delegando il carico a un thread pool gestito dall'OS. L'uso di un semaforo asincrono impedisce la saturazione della CPU, permettendo all'utente di continuare a ruotare la camera 3D mentre il motore calcola i cluster in background.

5. **Rendering Hardware-Accelerato WebGL**: Sfruttando l'accelerazione hardware tramite WebGL (Plotly.js), il sistema ottimizza il rendering di dataset sismici massivi, applicando mappature cromatiche dinamiche basate sulla profondità e trasparenze regolate dalla magnitudo. Questo approccio permette una gestione fluida delle nuvole di punti 3D, mantenendo un frame rate elevato anche durante le transizioni temporali e le rotazioni della camera.

6. **Interazione e Refinement Loop (State Persistence)**: L'architettura è intrinsecamente reattiva. Ogni interazione sulla GUI (cambio di cluster, soglia RANSAC) attiva una chiamata parziale via WebSocket. Il sistema implementa la persistenza dello stato (*uirevision*), assicurando che l'orientamento della camera e il livello di zoom vengano preservati tra un aggiornamento e l'altro, garantendo una navigazione fluida e priva di interruzioni cognitive.

3 Architettura di Sistema e Flussi di Comunicazione

L'architettura di *Seismic Viewer* è ispirata ai moderni sistemi di visualizzazione scientifica professionale, dove la potenza computazionale del backend è rigorosamente disaccoppiata dalla reattività dell'interfaccia.

3.1 Il Backend: FastAPI e Orchestrazione Asincrona

Il backend è sviluppato in Python 3.10+ e sfrutta il framework **FastAPI** per la gestione dei flussi di dati ad alta velocità. A differenza dei server sincroni tradizionali, l'intero ciclo di vita dell'applicazione è gestito da un *Event Loop* asincrono. Questo permette di gestire la comunicazione WebSocket in tempo reale mentre, in parallelo, vengono eseguiti calcoli ML pesanti.

Per garantire la massima stabilità, il backend implementa un sistema di **Thread Pool Isolation**. Le operazioni CPU-bound (come la ricalcolazione delle mesh geografiche o il fitting RANSAC) non vengono mai eseguite sul thread principale dell'Event Loop, ma

delegate a worker thread asincroni dedicati. Questa strategia elimina i colli di bottiglia computazionali e previene il congelamento dell'interfaccia utente (UI freeze), garantendo che i messaggi di sistema e le notifiche in tempo reale siano sempre fluidi e reattivi.

3.2 Il Frontend: Plotly.js e Ottimizzazione Hardware

Il motore di rendering è basato sulla libreria **Plotly.js**, che agisce come astrazione di alto livello sopra le API **WebGL**. Questo approccio permette di sfruttare l'accelerazione hardware della scheda video per il calcolo delle trasformazioni di camera (rotazione, zoom, pan) di decine di migliaia di oggetti ipocentrali. La gestione dello stato è centralizzata in un oggetto JavaScript globale, che sincronizza i filtri di magnitudo e tempo applicati dall'utente, garantendo che ogni aggiornamento sia coerente in tutte le viste (Mappa, 3D, Timeline).

3.3 Ingegneria del Dato: Streaming Binario e Memoria

Questo processo di **Binary Shift** rappresenta una risposta diretta alle criticità emerse nelle prime versioni del software, dove l'uso di JSON testuale causava saturazione della RAM e latenze di parsing inaccettabili. L'implementazione attuale non solo riduce il volume di dati trasferiti (eliminando la ridondanza semantica delle chiavi JSON), ma permette al browser di allocare la memoria in modo predittivo tramite `ArrayBuffer`. L'assenza di stringhe e oggetti intermedi riduce drasticamente l'attività del *Garbage Collector*, garantendo che l'applicazione possa gestire dataset sismici di intere decadi (oltre 200.000 eventi) mantenendo un utilizzo della RAM estremamente contenuto e una latenza di aggiornamento quasi impercettibile.

4 Metodologie Analitiche e Algoritmi

L'architettura analitica del software si fonda sulla capacità di estrarre strutture geometriche coerenti da dataset ipocentrali complessi, trasformando distribuzioni di punti sismici in modelli geologici.

4.1 Identificazione Strutturale tramite RANSAC e SVD

Per identificare i piani di faglia, il sistema implementa una pipeline di robustezza statistica che combina l'algoritmo **RANSAC** con la precisione del **Singular Value Decomposition**. La sfida principale risiede nel fatto che un dataset sismico contiene naturalmente molto rumore e terremoti che non appartengono a una faglia specifica.

Il processo segue una rigorosa logica matematica basata sull'analisi delle componenti principali (PCA):

1. **Campionamento Iterativo RANSAC**: Viene selezionato un sottoinsieme minimo di 3 punti $\{P_1, P_2, P_3\}$. Il piano candidato è definito dal vettore normale $\vec{n} = (P_2 - P_1) \times (P_3 - P_1)$.

2. **Ottimizzazione SVD**: Dato un insieme di N inliers identificati da RANSAC, la normale ottimale $\vec{n} = [a, b, c]$ viene calcolata tramite la decomposizione ai valori singolari. Si costruisce la matrice A di dimensione $N \times 3$ contenente i residui spaziali:

$$A = \begin{bmatrix} x_i - \bar{x} & y_i - \bar{y} & z_i - \bar{z} \end{bmatrix} \quad (1)$$

La normale corrisponde al vettore singolare destro associato al valore singolare più piccolo di A, garantendo la minimizzazione dei minimi quadrati della distanza ortogonale dei punti dal piano.

4.2 Geometria Strutturale: Strike, Dip e Dip Direction

Dalla normale del piano $\vec{n} = [n_x, n_y, n_z]$ (normalizzata e orientata in modo che $n_z > 0$), il software calcola i parametri fondamentali per la sismologia strutturale:

- **Dip (δ)**: L'angolo di inclinazione massima rispetto all'orizzontale, calcolato come $\delta = \arccos(n_z) \cdot \frac{180}{\pi}$. Un dip di 90° indica una faglia verticale.
- **Strike (ϕ)**: L'orientamento della traccia della faglia rispetto al Nord geografico. Viene determinato calcolando l'angolo del vettore di direzione della traccia sul piano orizzontale, utilizzando la funzione `atan2(n_x, n_y)` e normalizzando il risultato nel range $[0°, 360°]$.
- **Dip Direction**: La direzione verso cui il piano "immerge", calcolata come $(\phi + 90°)$ (mod 360).

4.3 Stabilità Temporale dei Cluster (Global Label Matching)

Nelle visualizzazioni animate 4D, il ricalcolo indipendente dei cluster per ogni frame causava inizialmente un "flickering" visivo inaccettabile (cambio di colore dei cluster tra un frame e l'altro). Per risolvere questo problema di instabilità cromatica, è stata sviluppata una logica di **Label Matching Globale**. Se un cluster nel frame attuale C_{curr} condivide una frazione significativa di eventi con un cluster nel frame precedente C_{prev}, ne eredita l'ID e il colore:

$$\text{Match} \iff \frac{|C_{curr} \cap C_{prev}|}{|C_{curr}|} > \text{Threshold} \tag{2}$$

Questo garantisce una stabilità visiva che permette all'utente di seguire l'evoluzione fisica e temporale di uno sciame sismico senza distrazioni cromatiche.

5 Security Architecture & Robustness

Sebbene il software operi in un ambiente di rete isolato (`localhost`), l'integrità del sistema è garantita da un'architettura di difesa in profondità. La comunicazione tra Python e il motore Chromium (tramite `pywebview`) è protetta da una serializzazione JSON deterministica che neutralizza ogni vettore di attacco **XSS** legato ai metadati dei file. Ogni stringa iniettata nel DOM viene processata come `textContent`, impedendo l'esecuzione di script malevoli annidati nei nomi dei file o nei metadati sismici.

Inoltre, la stabilità della CPU durante analisi massive è preservata da un sistema di **semafori asincroni** (`asyncio.Semaphore(1)`). Questo meccanismo di "critical section" asincrona assicura che il server FastAPI non tenti di eseguire più istanze simultanee di algoritmi intensivi (come il clustering spaziale o il fitting RANSAC) che potrebbero portare alla saturazione della RAM o al surriscaldamento della CPU, garantendo una UX fluida e priva di freeze applicativi anche su sistemi con risorse limitate.

6 Infrastruttura di Caricamento e Pre-calcolo

Una delle sfide principali del progetto è la gestione di file CSV che possono contenere decine di migliaia di record. Per evitare rallentamenti durante la navigazione, nel modulo `core/loader.py` è stata implementata una strategia di **Pre-computing**.

6.1 Normalizzazione Temporale

Il sistema non memorizza solo la data, ma calcola preventivamente il valore `dates_num`, che rappresenta il tempo trascorso in giorni dal primo evento osservato. Questo permette al motore di animazione JS di eseguire interpolazioni lineari istantanee senza ricalcolare date complesse a runtime.

6.2 Visual Data Packaging e Ottimizzazione delle Performance

Il caricamento dei dati non consiste in una semplice lettura di un file CSV, ma in un sofisticato processo di **Visual Pre-computation**. Questa tecnica, implementata nel metodo `precompute_visual_data`, è fondamentale per garantire che l'interfaccia JavaScript rimanga fluida (target 60 FPS) anche durante la gestione di dataset massivi.

Il "pacchetto" creato da Python comprende tre elementi critici:

- **Pre-rendering delle Stringhe di Hover**: Per ogni evento sismico, il backend genera preventivamente una stringa HTML complessa contenente data, magnitudo e coordinate. Spostando questa operazione di concatenazione di stringhe da JavaScript a Python, si elimina un enorme carico computazionale dal thread principale del browser, prevenendo micro-lag durante l'interazione con la mappa.

- **Aggregazione Temporale Flessibile**: Per supportare le diverse esigenze analitiche, il sistema pre-calcola indici temporali che abilitano due distinte modalità di visualizzazione:

  - *Cumulativa*: analisi dell'evoluzione storica e volumetrica totale degli eventi fino alla data selezionata.
  - *Finestra Temporale Mobile*: isolamento di uno specifico intervallo di tempo (es. scivolamento di una finestra di 10 giorni) fondamentale per identificare i trend di migrazione sismica e i pattern di attivazione delle faglie nel tempo.

  Questo approccio strutturato permette al ricercatore di alternare istantaneamente tra le diverse modalità senza ricalcoli onerosi, garantendo un'esplorazione dinamica e fluida del dataset.

- **Metadata Range Synchronization**: Vengono calcolati preventivamente i limiti minimi e massimi di tutte le variabili (magnitudo, profondità, tempo). Questo assicura che le scale cromatiche e i limiti dei filtri dinamici siano sincronizzati istantaneamente al caricamento, evitando flicker o ricalcoli asincroni lato client.

Contemporaneamente, il sistema esegue una normalizzazione spaziale, mappando la profondità sismica sul sistema cartesiano invertendo i segni ($z < 0$), assicurando che l'ipocentro sia correttamente localizzato al di sotto della superficie terrestre.

7 Motore di Visualizzazione e Geospazialità

La componente geografica è gestita dal modulo `core/geo_utils.py`, che si occupa di tradurre il mondo reale in una scena 3D coerente.

7.1 Formalizzazione della Proiezione di Mercatore

Il software implementa una proiezione di Mercatore **inversa** custom, necessaria per mappare correttamente la griglia di tasselli cartografici scaricati sulla geometria 3D. La formula utilizzata per la conversione delle coordinate piane (Y) in coordinate sferiche (Latitudine) è descritta dalla seguente relazione:

$$\phi = \arctan\left(\sinh\left(\pi\left(1 - \frac{2 \cdot y}{n}\right)\right)\right)$$

dove n rappresenta lo zoom level della mappa e y la coordinata normalizzata del tassello. Questo calcolo, eseguito nel modulo `core/geo_utils.py`, assicura la perfetta sovrapposizione tra la mesh del terreno e la posizione reale degli ipocentri sismici.

7.2 Algoritmi di Clustering e Machine Learning

Per l'analisi delle strutture sismogenetiche, il sistema integra algoritmi di *clustering spaziale* calibrati per il calcolo scientifico:

- **MiniBatchKMeans**: una variante ottimizzata dell'algoritmo K-Means che utilizza sotto-insiemi casuali del dataset per aggiornare i centroidi. Questa scelta è critica per *Seismic Viewer* poiché permette di mantenere la reattività dell'interfaccia (latenza $< 500ms$) anche quando l'utente manipola il numero di cluster in tempo reale su dataset di grandi dimensioni.
- **DBSCAN (Density-Based Spatial Clustering)**: implementato per identificare strutture sismiche a morfologia complessa (non sferica). Il parametro *Epsilon* viene mappato su distanze euclidee reali (km), permettendo di isolare automaticamente il rumore di fondo dai *clusters* densi che rappresentano i piani di faglia attivi.

8 Sottosistema di Analisi Statistica e ML

Il sistema non è solo un visualizzatore, ma un analista automatico guidato da algoritmi di Machine Learning situati in `engine.py`.

8.1 Clustering Centroidale (MiniBatchKMeans)

Viene utilizzato l'algoritmo **MiniBatchKMeans** per raggruppare gli eventi in k zone. L'uso della versione "MiniBatch" è una scelta progettuale precisa: essa processa i dati in blocchi casuali, garantendo che anche con 100.000 eventi il calcolo avvenga in meno di un secondo, mantenendo al contempo un'altissima aderenza ai centroidi reali.

8.2 Analisi Densimetrica (DBSCAN)

Per identificare sciami sismici di forma non circolare (tipico delle faglie lineari), viene implementato il **DBSCAN**. Questo algoritmo non richiede il numero di cluster in input, ma lavora sui parametri di densità (Epsilon e MinSamples), permettendo di isolare i terremoti "rumore" (isolati) dalla sequenza principale.

8.3 Rappresentazione dei Volumi (Convex Hull)

Per rendere visibile l'influenza spaziale di un cluster, il software calcola il **Convex Hull** (Inviluppo Convesso). Utilizzando l'algoritmo Quickhull, viene generata una mesh 3D che "avvolge" tutti i punti del cluster, dando la percezione del volume di roccia interessato dalla sequenza sismica.

9 UX/UI Design & Interaction Model

L'interfaccia utente è progettata per essere "viva" e reattiva, utilizzando i canoni estetici del **Glassmorphism**.

9.1 Filosofia Glassmorphism

I menu e le dashboard utilizzano sfondi semi-trasparenti con effetto di sfocatura dello sfondo. Questa scelta non è puramente estetica, ma risponde a una necessità ergonomica: permette di sovrapporre i pannelli di controllo ai dati sismici senza ostruire completamente la percezione della profondità della scena 3D sottostante, riducendo l'affaticamento visivo durante le sessioni di analisi prolungate.

9.2 Sistema di Auto rotazione e Controlli di Navigazione

Per massimizzare l'usabilità e la precisione dell'analisi spaziale, il sistema integra funzionalità di navigazione avanzata, risolvendo la criticità del reset della camera durante i filtraggi dei dati:

- **Persistenza della Camera (`uirevision`)**: *Seismic Viewer* mantiene lo stato di zoom e orientamento dell'utente anche quando il dataset viene aggiornato (p.es. al cambio di magnitudo o data), garantendo un'esplorazione priva di interruzioni.
- **Auto rotazione**: permette di osservare la nube di punti sismici da ogni angolazione, facilitando l'identificazione di pendenze o allineamenti di faglie non visibili da una prospettiva fissa.
- **Selettori di Prospettiva**: pulsanti dedicati per il cambio istantaneo del punto di osservazione (vista dall'alto o laterale), permettendo di passare rapidamente dall'analisi planimetrica a quella del profilo ipocentrale.

9.3 Modello di Interazione Asincrona e Gestione Multithreading

L'efficienza della *User Experience* (UX) all'interno di Seismic Viewer è garantita dall'adozione di un paradigma di interazione asincrona, volto a mitigare le latenze di risposta

del thread principale durante l'esecuzione di task ad alto onere computazionale (p.es. la generazione di mesh ad alta risoluzione o l'analisi statistica tramite clustering di dataset massivi).

L'architettura del sistema si fonda su una gestione strutturata della concorrenza articolata in tre livelli:

- **Delegazione Computazionale (Worker Threads)**: Ogni richiesta generata dal frontend viene intercettata dal modulo `core/bridge.py` e instradata su un thread indipendente mediante la libreria `threading`. L'implementazione di thread di tipo "daemon" assicura la sincronizzazione del ciclo di vita dei processi in background con quello dell'applicazione principale, garantendo una corretta deallocazione delle risorse al termine della sessione.

- **Propagazione dello Stato via Callback**: Durante l'esecuzione dei processi asincroni, il backend Python comunica costantemente lo stato di avanzamento al motore di rendering. Tale interscambio avviene tramite l'invocazione del metodo `evaluate_js`, che inietta istruzioni JavaScript all'interno del motore Chromium per l'aggiornamento dinamico del *Progress Hub*. Questo meccanismo di feedback continuo minimizza l'incertezza informativa per l'utente, mantenendo alta la percezione di reattività del sistema.

- **Sincronizzazione Event-Driven**: Per preservare l'integrità del rendering, le transizioni di stato tra i diversi moduli dell'applicazione sono regolate da meccanismi di temporizzazione (`threading.Timer`). Questo approccio consente di regolare la latenza necessaria affinché il server HTTP locale completi le operazioni di I/O sui file temporanei, assicurando transizioni atomiche e prive di anomalie visive.

10 Gestione Cache e Performance

Per garantire l'usabilità professionale, Seismic Viewer implementa tecniche sofisticate di gestione delle risorse nel package `core/`.

10.1 Caching Determinista MD5

Le mesh del terreno e i dati cartografici vengono calcolati una sola volta per ogni area geografica. Il sistema genera un hash **MD5** deterministico basato sulle coordinate di bounding-box e sul livello di zoom. Se un hash corrispondente è presente nel database locale (`resources/.map_cache`), la risorsa viene caricata istantaneamente tramite il protocollo binario **Pickle**. Questo approccio riduce i tempi di startup della visualizzazione da diversi secondi (necessari per il download e la tassellazione) a pochi millisecondi, rendendo il software pronto all'uso immediato anche in condizioni di connettività limitata.

10.2 Ottimizzazioni di Memoria (LRU)

Le funzioni di generazione mesh utilizzano un decoratore `lru_cache` (Least Recently Used) a livello di sistema. Questo assicura che se l'utente torna a una visualizzazione precedente, essa venga caricata istantaneamente dalla RAM anziché dal disco o dal web.

11 Guida all'Uso e Casi d'Uso

L'applicazione Seismic Viewer è stata progettata per supportare il ricercatore in diverse fasi dell'analisi sismologica. Di seguito vengono descritte le principali modalità operative e di visualizzazione integrate:

- **Analisi Planimetrica 2D**: Fornisce una vista planimetrica classica della distribuzione degli epicentri. È lo strumento primario per una rapida valutazione dell'estensione geografica della sequenza e per l'allineamento con la cartografia stradale o urbana.
- **Modellazione Volumetrica 3D**: Integra i dati ipocentrali con la morfologia digitale del terreno. Questa vista permette di apprezzare la profondità degli eventi e l'inclinazione delle strutture sismogenetiche, visualizzando i terremoti al di sotto della mesh superficiale.
- **Simulazione Temporale 4D**: Attraverso la modalità di animazione, il sistema permette di osservare la propagazione spazio-temporale dell'energia. Questa analisi è fondamentale per distinguere la migrazione dei fluidi o le attivazioni a catena di diverse faglie nel tempo.
- **Analisi Avanzata tramite Clustering**: Implementa algoritmi di clustering spaziale per l'identificazione automatica di cluster sismici. Tale approccio facilita la discriminazione analitica di differenti sotto-sequenze e l'identificazione di strutture sismogenetiche indipendenti, basandosi su rigorosi criteri di densità spaziale e temporale dei dati ipocentrali.

Il workflow tipico prevede il caricamento di un dataset (es. Terremoti dei Campi Flegrei), l'analisi preliminare in 2D e il successivo approfondimento volumetrico e statistico per caratterizzare la geometria della sorgente sismica.

12 Gestione degli Errori e Robustezza del Software

Nello sviluppo di software scientifico, la gestione delle eccezioni è fondamentale per evitare la perdita di dati o interruzioni brusche durante lunghe sessioni di analisi. *Seismic Viewer* implementa un sistema di gestione errori a più livelli:

- **Resilienza di Rete**: Il modulo `online_importer` è progettato per garantire la continuità operativa anche in presenza di instabilità dei servizi remoti. Invece di annullare l'intero caricamento al primo errore di connessione, il sistema isola le richieste per singoli blocchi annuali. Se i dati di un particolare anno non sono disponibili (es. per un downtime temporaneo dei server INGV), il software segnala l'anomalia ma prosegue con il recupero delle restanti parti del dataset. Questo approccio assicura che l'analista possa comunque disporre della quasi totalità dei dati, minimizzando l'impatto di malfunzionamenti esterni sulla sessione di lavoro.
- **Integrità del Caching**: Se un file mesh (`.pkl`) risulta corrotto, il sistema intercetta l'eccezione durante la fase di *unpickling*, elimina il file danneggiato e avvia la rigenerazione automatica della risorsa.

- **Validazione dei Parametri ML**: Gli algoritmi di clustering spaziale verificano preventivamente se il numero di punti è sufficiente ($N > \text{MinPts}$). In caso contrario, l'analisi viene interrotta con un feedback non bloccante per l'utente, prevenendo il crash del motore di calcolo.

13 Analisi delle Sfide Tecniche e Risoluzione dei Problemi

L'attuale stato di maturità del software è il risultato del superamento di diverse sfide ingegneristiche complesse, emerse durante le fasi di test su dataset reali.

13.1 Persistenza dello Stato e "Camera Reset Issue"

Una delle difficoltà iniziali più sentite dagli utenti riguardava il reset automatico della camera 3D ad ogni aggiornamento dei dati. Filtrare una sequenza sismica per magnitudo causava il ritorno della vista alla posizione predefinita, interrompendo l'analisi spaziale.

- **Soluzione**: È stata implementata la proprietà `uirevision` di Plotly.js nel modulo di rendering. Questo ha permesso di disaccoppiare lo stato della camera dallo stato dei dati, consentendo un'esplorazione continua e fluida anche durante manipolazioni massive del dataset.

13.2 Stabilità Cromatica nelle Animazioni 4D

Durante le simulazioni temporali, il ricalcolo dei cluster per ogni frame causava il cambiamento casuale dei colori associati ai gruppi di terremoti (flickering), rendendo impossibile seguire visivamente l'evoluzione di uno sciame specifico.

- **Soluzione**: È stato sviluppato un algoritmo di **Global Label Matching**. Il sistema ora confronta l'intersezione degli eventi tra il frame T e $T-1$, riassegnando l'ID del cluster precedente se la similitudine supera una soglia critica, garantendo una coerenza visiva temporale assoluta.

13.3 Ottimizzazione dei Tempi di Caricamento (Binary Shift)

L'uso iniziale del formato JSON per il trasferimento di oltre 50.000 eventi causava lag pesanti e, in alcuni casi, il crash del motore di rendering a causa della frammentazione della memoria.

- **Soluzione**: Il sistema è stato migrato a un'architettura di **Binary Streaming**. I dati vengono ora impacchettati in Python come *byte-stream* e decodificati direttamente in memoria JavaScript. Questa modifica ha ridotto i tempi di caricamento del 90% e l'uso della RAM del 75%, come documentato nei commit di ottimizzazione delle performance.

13.4 Resilienza alle Sorgenti Dati (INGV ed Error handling)

L'integrazione con il portale GOSSIP dell'INGV ha presentato sfide legate all'instabilità delle connessioni e alla varietà dei formati data (US vs EU).

- **Soluzione**: È stato introdotto un **Online Importer** resiliente che implementa il *check-summing* dei dati e la normalizzazione automatica dei formati data in un formato standard europeo (DD-MM-YYYY), garantendo la compatibilità con i sistemi di monitoraggio locali.

14 Conclusioni e Prospettive Evolutive

In conclusione, *Seismic Viewer* si configura come una struttura modulare e altamente scalabile dedicata alla ricerca geofisica avanzata. L'architettura del software è già predisposta per accogliere algoritmi di intelligenza artificiale di nuova generazione, come le **Graph Neural Networks (GNN)**, per l'identificazione di pattern di propagazione dello stress sismico non rilevabili con i metodi statistici classici. Inoltre, l'integrazione futura di modelli **LSTM** (*Long Short-Term Memory*) consentirà di stimare l'evoluzione degli sciami sismici, fornendo un supporto decisionale quantitativo fondamentale per la protezione civile.

Il disaccoppiamento logico tra i processi di elaborazione del backend e le componenti di visualizzazione del frontend abilita l'incorporazione di questi complessi modelli predittivi senza degradare la reattività del sistema. *Seismic Viewer* riesce così a coniugare un elevato rigore scientifico con un'interfaccia utente ottimizzata secondo i principi della moderna interazione uomo-macchina, ponendosi come un efficace strumento di supporto alla ricerca sismologica e alla mitigazione del rischio geologico.

Riferimenti bibliografici

[1] Istituto Nazionale di Geofisica e Vulcanologia (INGV). *Osservatorio Vesuviano - Portale GOSSIP*. `https://terremoti.ov.ingv.it/gossip/`

[2] Plotly Technologies Inc. *Collaborative data science*. Montreal, QC, 2015. `https://plot.ly`

[3] Wes McKinney. *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, 2010.

[4] Pedregosa et al. *Scikit-learn: Machine Learning in Python*. JMLR 12, pp. 2825-2830, 2011.